Human Report

Building this final project alone was challenging. I will highlight some initial problems I faced, and then discuss some of my major attempts that led to my final attempt.

Implementing Neural Network

My first challenge was converting my parameterized value function into a neural network effectively and without bugs. I used an example shown in class as a starting point. For the most part, the example shown in class worked incredibly well. The only improvement I added was a set of inputs for the negative places on the board. I also add inputs to the code for certain features, particularly player one and two's 2 and 3 in a row.

Stumbling with Vectorization

One problem that followed me from assignment 5 was figuring out what was wrong with my vectorization game loop. I debugged for too many hours to determine exactly what was wrong with my code. Unfortunately, I could not get it working, and decided to pivot to doing a single game at a time. If I were to do this assignment again, I would focus on a single game submission from the beginning and save all of the debugging I had to do.

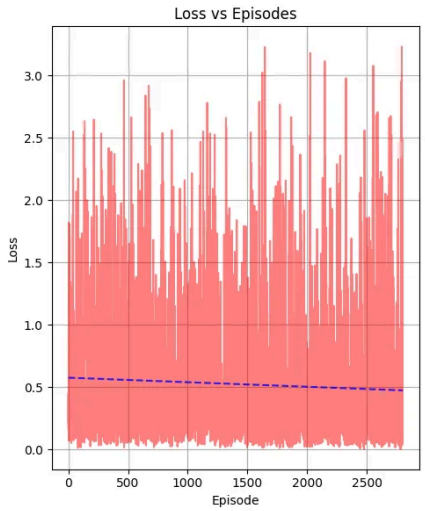
Attempts Gallery

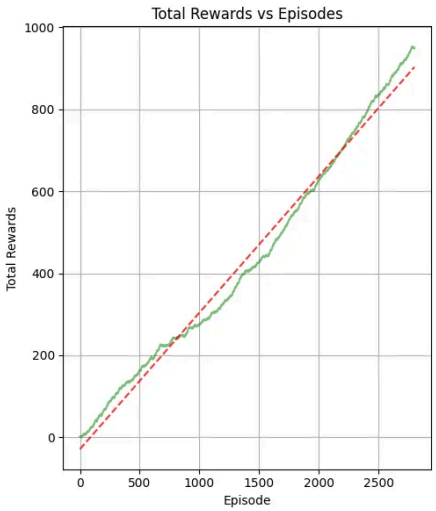
This is to quickly highlight a few of my attempts at training the AI. I tested some attempts with the leaderboard. Each attempt has a plot of the average loss of the value function and total rewards over time.

Attempt 1.

For this attempt, I did not have any Q function implemented yet, so no roll again functionality. But this was my first attempt using the single game loop, and I was

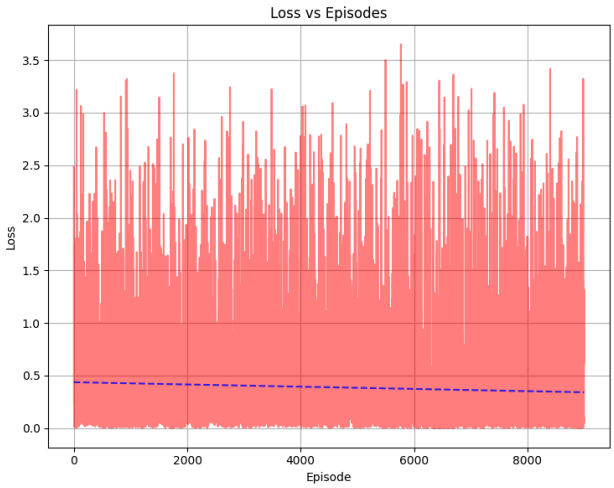
shocked by how my value function neural network began to learn instantaneously. This was trained only against the Random AI. I did not submit this version to the leaderboard.

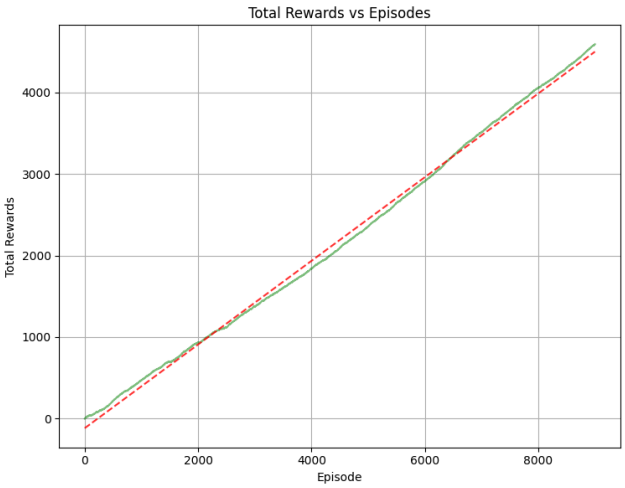




Attempt 2.

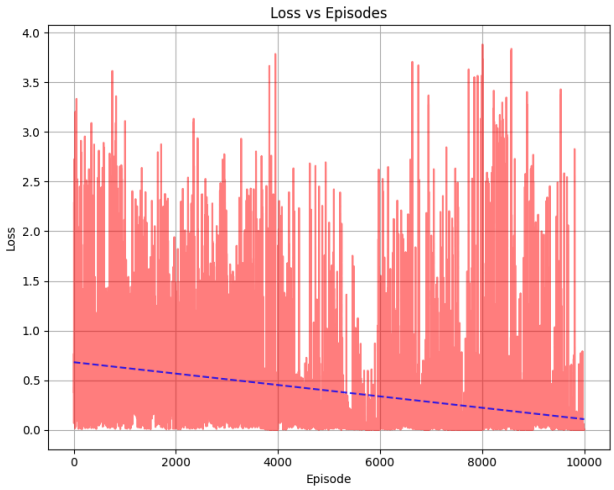
For my next attempt, I implemented the Q-function neural network. This neural network learns alongside the value function, but learns on the set of actions taken rather than just board states. This attempt was trained on the random AI as well, however, there was a major bug in this attempt. The Q function was not using its own loss function, grad and parameters, I accidentally set it to use the value function parameters. This greatly handicapped the AI and effectively made the Q function useless. From the leaderboard, this attempt got 24% against the bot and 74% against the random.

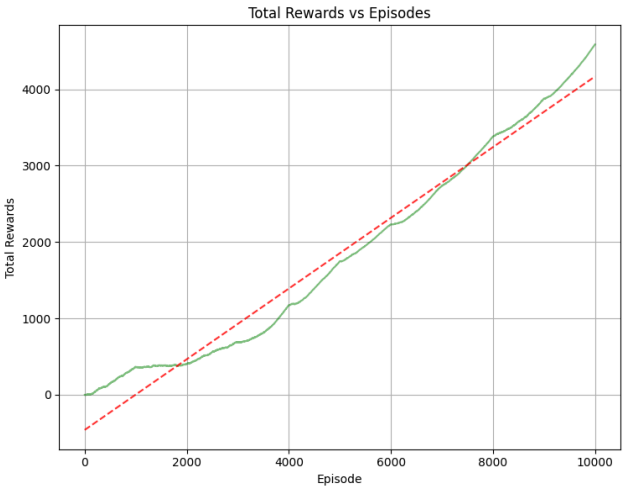




Attempt 3.

After I ironed out the bugs with my Q function, I switched to train my AI against itself. This allows my AI to find holes in its strategy and learn how to defend itself. I set it up to play 1000 games against an old version of itself, and then update the old version to the latest. You can see the effect of this on the total rewards graph, where the AI struggles, learns to get better, and then struggles all over again. This approach improved its performance against the bot, now 26%, but somehow decreased its performance against the random AI, now 69%.





Final Approach

My final approach used Monte-Carlo learning and neural networks for both the Q function and the value function. I pulled out all of the stops for my final attempt to try to make an AI that could reach the top 3 on the class leader board. The following are all of the tactics I used to get optimal results.

Q-function neural network

Developing my Q-function was quite difficult. I approached it from several different angles. Initially, I attempted to add a separate output to the value function that would determine roll again, however, I found that it complicated my code, and the AI couldn't learn to use it just based on training by value. I also looked into developing a typical Q[S, A] matrix, but I realized the state-action space would be way too large to maintain in this case. This led me to parameterize the Q function, but I decided that if I am already going to parameterize it, I may as well learn how to make it into a neural network. My Q function neural network is identical to my value network except it has 2 outputs: one for each action (stop, roll again). To train it, I ensured to keep track of what actions were taken from each board state, and created a loss function that calculates the loss for each action in a one-hot way. This means that I can reinforce each action separately from the other.

Playing against itself

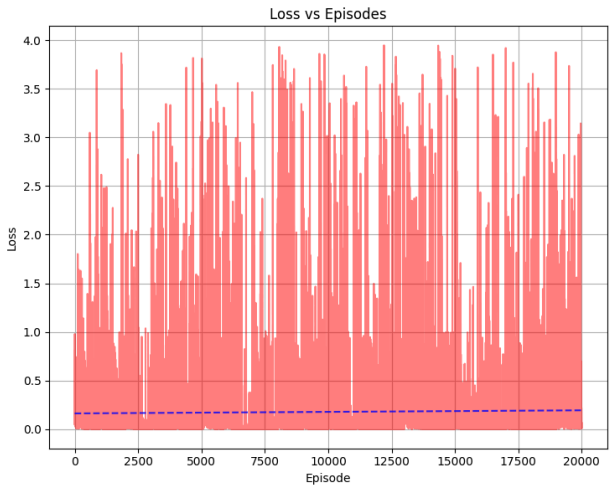
One way I improved training vs the random AI is by training my AI against itself. The strategy I took was to allow my AI to play ~100 games against its former self; if the average loss over the last 50 games comes to less than 0.1, set the opponent to the new parameters. If the average loss is still over 0.1, continue to play games until it isn't. This approach allows my AI enough time to effectively learn against itself and better itself, more than simply running it on a set number of games.

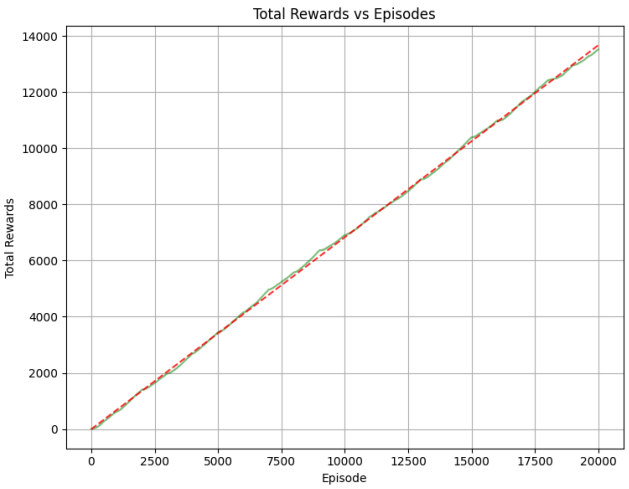
Multiple game boards

Another way I improved training was by creating permutations of every board my AI encounters. This is easy to do as Pig Tac Toe is completely symmetrical. To achieve this, I stored every rotation of each board my AI comes across as well as storing the flipped version of every rotation. Every board my AI encounters generates 8 permutations, meaning 8 data points to train on. This greatly improved my training performance and ensured my AI received a deep understanding of the game.

Features

The final way I improved my AI was by adding custom features to my neural network inputs. This isn't necessary, as the neural network can learn these features on its own based solely on the game boards it's seen. However, adding features helps the neural network identify patterns more quickly and greatly improves my training speed. The features I included were all potential two and three-in-a-row for player 1 and player 2. This had the greatest effect on learning effectiveness out of all my choices, and drastically improved the speed at which my AI learned.





I was able to achieve 38% against the bot and 75% against the random AI, securing my spot among the top 3 submissions in the class. One thing I should have done is record the loss of the Q function as well. This was a fun project to build, and a great way to finish up the class. In general, I hope to apply reinforcement learning skills in my side projects and career going forward.